

ECE PARIS

Grande École de l'Ingénierie Numérique

Campus Eiffel - 10 rue Sextius Michel, 75015 Paris

PROJET DE SUBSTITUTION AU STAGE

Cycle Bachelor - 3ème année

*Conception et développement d'un détecteur hybride de ransomware
(heuristique et apprentissage automatique) avec console SOC multi-analystes*

Étudiants - groupe de trois membres

Membre 1 : KOUASSI Yao Franck-Arsène

Membre 2 : FOKOU Deffo Franck Emerite

Membre 3 : ODOULAMI Philippe Leopold

Spécialisation : Cybersécurité & réseaux

Session : FALL 2026

Établissement d'accueil

Nom : ECE Paris (École Centrale d'Électronique)

Adresse : 10 rue Sextius Michel, 75015 Paris, France

Engagement de confidentialité (NDA) : non

Tuteur académique : DERRAZ Foued

Téléphone du tuteur : +33 1 44 39 25 65

Email du tuteur : fderraz@ece.fr

Signature du tuteur :

Période du projet : du 5 juillet 2026 au 20 août 2026

Description de la mission

Conception, implémentation, test et documentation d'une plateforme EDR de laboratoire capable de détecter des comportements de rançongiciel sur des postes Windows, d'y répondre automatiquement et de permettre à plusieurs analystes de superviser le parc depuis une console web partagée.

Contents

Conception et développement d'un détecteur hybride de ransomware (heuristique et apprentissage automatique) avec console SOC multi-analystes	1
REMERCIEMENTS.....	4
INTRODUCTION.....	5
I- MISSIONS ET TÂCHES CONFIÉES	7
Contexte et objectifs du projet (résumé version française)	7
Contexte et objectifs du projet (résumé version anglaise).....	7
Organisation du projet et méthodologie.....	8
Planning prévisionnel.....	9
II- REALISATION DU PROJET	9
a. Infrastructure et environnement technique.....	9
b. Pipeline de collecte (Sysmon et Winlogbeat)	10
d. Extraction de caractéristiques et baseline.....	11
f. Apprentissage automatique (Random Forest et LSTM)	12
g. API, réponse active et agent de riposte	13
h. Console SOC multi-analystes et déploiement.....	14
i. Tests et validation du pipeline	15
j. Bilan des missions	16
III- CONCLUSION ET PERSPECTIVES	18
IV- BIBLIOGRAPHIE / WEBOGRAPHIE	19
V- ANNEXES.....	21
Annexe 1 : Schéma d'architecture de la plateforme	21

REMERCIEMENTS

Les trois membres du groupe tiennent à remercier l'ECE Paris pour l'encadrement académique dont on a bénéficié tout au long de ce projet de substitution de stage. L'école nous a fourni un cadre exigeant, des ressources documentaires et la possibilité de mener un travail technique d'ampleur, comparable à une mission professionnelle de cybersécurité opérationnelle.

les trois membres se remercient mutuellement. La répartition des rôles, pipeline de collecte, moteurs de détection, plateforme SOC, n'a fonctionné que grâce à une coordination quotidienne, à des revues de code croisées et à une exigence partagée sur la qualité du livrable final.

INTRODUCTION

C'est dans le cadre de notre dernière année de bachelor cybersécurité et réseaux à l'ECE Paris qu'on a eu à réaliser, de juillet à août 2026, un projet de substitution de stage. Cette modalité, prévue par le cursus lorsque le stage en entreprise n'est pas la voie retenue ou n'est pas disponible dans les délais, impose de conduire une mission technique complète, documentée et soutenue, avec le même niveau d'exigence qu'une immersion professionnelle.

Le sujet retenu, ransomware detector, s'inscrit dans une problématique contemporaine de la blue team. Les rançongiciels demeurent l'une des menaces les plus destructrices pour les organisations : ils chiffrent les données, sabotent les copies de sauvegarde et réclament une rançon. Les antivirus fondés uniquement sur des signatures statiques sont souvent contournés par des variantes polymorphes ou des attaques sans fichier. L'équipe a donc choisi une approche comportementale et hybride : observer ce que fait réellement un poste windows, en déduire des indicateurs mathématiques, puis croiser un moteur de règles et un modèle d'apprentissage automatique avant d'ordonner une riposte.

Le projet ne s'est pas arrêté à un prototype de détection. Nous avons construit une plateforme de supervision utilisable par plusieurs analystes : comptes et habilitations, base de données partagée, console web en temps réel, journal d'audit et déploiement par conteneurs. L'ensemble a été testé dans un laboratoire isolé, à l'aide d'un simulateur inoffensif reproduisant des techniques décrites dans le référentiel MITRE ATT&CK.

Le présent rapport décrit, l'analyse les missions techniques confiées au groupe, les difficultés rencontrées et les résultats obtenus, avant de conclure sur le bilan de l'expérience et les perspectives d'évolution de la solution.

I- MISSIONS ET TÂCHES CONFIÉES

Contexte et objectifs du projet (résumé version française)

La mission qui nous a été confiée s'inscrit dans une question opérationnelle simple à formuler, difficile à résoudre : comment détecter un rançongiciel sur un poste windows sans se fier uniquement à une signature de fichier, puis comment y répondre assez vite pour limiter le chiffrement, tout en donnant à une équipe d'analystes une vision partagée et traçable de l'incident ?

Cette question est au cœur des EDR (Endpoint Detection and Response) modernes. Un antivirus classique compare un échantillon à une base de hachages. Une EDR observe des comportements : création massive de fichiers, entropie anormale des noms, processus enfants inhabituels, tentatives d'effacement des clichés volume shadow copy, communications vers un serveur de commande et contrôle. Le projet a consisté à reconstruire, à l'échelle d'un laboratoire, cette chaîne : capteur, normalisation, vectorisation, décision hybride, riposte, supervision.

Les objectifs définis en début de projet étaient les suivants : mettre en place un laboratoire isolé reproductible ; collecter des événements sysmon pertinents ; extraire un vecteur de caractéristiques par fenêtre temporelle ; apprendre une baseline par machine ; implémenter un moteur de règles et un modèle d'apprentissage automatique ; déclencher une réponse active (arrêt de processus, isolation réseau) ; doter le tout d'une console SOC multi-analystes sécurisée ; déployer la plateforme avec docker ; documenter et valider par des tests et une démonstration live.

Contexte et objectifs du projet (résumé version anglaise)

The mission entrusted to us addresses an operational question that is simple to formulate but difficult to solve: how to detect ransomware on a Windows workstation without relying solely on file signatures, then how to respond quickly enough to limit encryption, while providing an analyst team with a shared and traceable view of the incident?

This question lies at the heart of modern EDRs (Endpoint Detection and Response). A traditional antivirus compares a sample against a hash database. An EDR observes behaviours: mass file creation, abnormal filename entropy, unusual child processes, attempts to delete Volume Shadow Copies, communications with a command-and-control server. The project involved reconstructing this chain at laboratory scale: sensor, normalization, vectorization, hybrid decision-making, response, and supervision.

The objectives defined at the start of the project were as follows: set up a reproducible isolated laboratory; collect relevant Sysmon events; extract a feature vector per time window; learn a per-machine baseline; implement a rule engine and a machine learning model; trigger an active response (process termination, network isolation); equip the platform with a secure multi-analyst SOC console; deploy the platform using Docker; document and validate through testing and a live demonstration.

Organisation du projet et méthodologie

Contrairement à un stage individuel en entreprise, le travail a été mené à trois, avec une répartition des modules et des intégrations fréquentes. Cette configuration nous a forcé à définir des contrats d'interface (formats JSON, routes d'API, schémas de base) avant d'avancer chacun de notre côté. Le dépôt GitHub commun a servi de source de vérité. Les imprévus d'intégration, permissions, tokens, fenêtres temporelles, concurrence d'accès à la base, ont occupé une part importante du calendrier, ce qui reflète la réalité d'un projet de cybersécurité plus que celle d'un exercice de cours.

Planning prévisionnel

Période	Missions réalisées
5 – 11 juillet 2026	Phase 1 : dépôt Git, VM windows, sysmon, winlogbeat, snapshot propre
12 – 19 juillet 2026	Phase 2 : parseur, extracteur de features, baseline, endpoint d'ingestion
20 – 26 juillet 2026	Phase 3 : moteur de règles, simulations, calibration des seuils
27 juillet – 4 août 2026	Phase 4 : dataset, random forest, LSTM, choix du modèle retenu
5 – 10 août 2026	Phase 5 : API complète, agent de riposte, isolation, pipeline de bout en bout
11 – 16 août 2026	Phase 6 : console SOC, PostgreSQL, docker, tests d'intégration
17 – 20 août 2026	Phase 7 : documentation, rapport, répétition de la démonstration

II- REALISATION DU PROJET

a. Infrastructure et environnement technique

Avant d'écrire le moindre algorithme de détection, l'équipe a dû construire un laboratoire dans lequel un rançongiciel, même simulé ne pourrait pas s'échapper. Cette étape, souvent sous-estimée, a conditionné toutes les décisions suivantes : adressage, jetons, chemins de logs, et jusqu'au mode de déploiement du serveur.

L'infrastructure repose sur un PC hôte Windows, équipé de VMware workstation, et d'une machine virtuelle windows 10 jouant le rôle du poste victime. Le réseau retenu est le mode Host-Only VMnet1 : la machine virtuelle communique avec l'hôte, pas avec internet. Cette isolation simule un laboratoire de malware responsable. Dans la configuration documentée, la machine virtuelle porte l'adresse 192.168.10.10 ; l'hôte s'est vu attribuer, selon les postes, 192.168.10.1 ou 192.168.10.2. L'équipe a appris à ses dépens que cette adresse n'est pas universelle : chaque membre devait relire ipconfig sur sa propre machine avant de pointer winlogbeat vers le serveur.

Un snapshot VMware nommé d'état propre a été pris avant toute exécution du simulateur. Sans ce point de restauration, chaque test aurait laissé des fichiers « chiffrés » et des artefacts

de persistance, rendant les expériences incomparables. Cette discipline, banale en analyse de malware, a structuré le rythme des démonstrations.

Côté serveur, le stack final tient en trois conteneurs docker : PostgreSQL 16 pour la persistance, une API FastAPI pour la détection et la décision, un frontal nginx servant la console react. Le choix de docker compose n'est pas cosmétique. Il garantit qu'un jury, un camarade ou un futur recruteur peut reproduire la plateforme sans installer python ni node.js sur l'hôte. Les secrets (mot de passe de base, secret de session, jeton d'agent, mot de passe d'amorçage) vivent dans un fichier .env jamais versé dans Git.

b. Pipeline de collecte (Sysmon et Winlogbeat)

Une EDR qui ne voit rien ne décide rien. La première brique technique a donc été la collecte d'événements windows à un niveau de détail suffisant pour reconstruire un comportement de rançongiciel.

Sysmon, outil de la suite Microsoft sysinternals, enrichit le journal d'événements windows bien au-delà des journaux de sécurité standards. L'équipe l'a déployé avec la configuration communautaire SwiftOnSecurity, puis a restreint le traitement à quatre identifiants d'événements : la création de processus (Event ID 1), la connexion réseau (Event ID 3), la création de fichier (Event ID 11) et la suppression de fichier (Event ID 23). Ces quatre familles suffisent à observer le lancement d'un binaire malveillant, un appel vers un serveur de commande, l'explosion de fichiers chiffrés et, le cas échéant, l'effacement des originaux. Les autres événements sysmon, trop nombreux, auraient noyé le pipeline.

Winlogbeat, agent léger d'Elastic en version 8.18.3, lit ces journaux et les expédie au serveur d'analyse au format json. Dans le laboratoire, il pointe vers le port 8000 de l'hôte et s'authentifie avec le même jeton d'agent que celui déclaré dans le fichier d'environnement du serveur. Sans ce jeton, n'importe quelle machine du réseau virtuel pourrait injecter de faux événements, fausser la baseline ou intercepter un ordre d'arrêt destiné à un autre poste.

Le parseur sysmon, écrit en python, aplanit la structure imbriquée de Winlogbeat (winlog.event_data) en dictionnaires normalisés : horodatage, action, nom et identifiant de processus, processus parent, fichier cible, adresse IP et port. Cette normalisation est le contrat entre la collecte et tout le reste du système. Les tests unitaires du parseur, alimentés par des extraits de journaux synthétiques, ont évité que le moindre changement de champ elastic ne casse silencieusement la détection.

c. Difficultés rencontrées

Une difficulté récurrente a concerné le réseau du laboratoire. Un winlogbeat mal pointé, un pare-feu windows trop fermé ou un jeton recopié d'un camarade produisait le même symptôme visible : des terminaux vides dans la console. L'équipe a documenté ces cas dans le guide de reproduction, car ils représentent, en pratique, plus de temps perdu que les algorithmes.

d. Extraction de caractéristiques et baseline

Les algorithmes d'apprentissage automatique ne lisent pas des journaux json. L'équipe a donc conçu un extracteur qui agrège les événements par fenêtres glissantes de dix secondes et calcule un vecteur de douze dimensions. Dix secondes constituent un compromis empirique : assez court pour interrompre un chiffrement avant qu'il n'emporte tout le disque, assez long pour distinguer une rafale malveillante d'un enregistrement bureautique isolé.

Parmi ces douze caractéristiques figurent le nombre de fichiers créés, supprimés ou renommés, la diversité des extensions, l'entropie de shannon des noms de fichiers, le nombre de processus créés, le nombre de processus enfants, la profondeur de l'arbre des processus, le volume de connexions, le nombre d'adresses ip distinctes, les connexions vers l'extérieur et les requêtes DNS. L'entropie joue un rôle particulier. Un nom légitime tel que rapport_annuel.docx se situe autour de 3,0 à 3,5. Un nom généré par un algorithme de chiffrement dépasse souvent 5,5. Une hausse brutale de cette métrique sur une fenêtre courte est un indice fort d'infection, indépendamment de l'extension affichée.

Le moteur de baseline observe ensuite environ cent secondes d'activité normale, dix fenêtres et calcule, pour chaque caractéristique, la moyenne et l'écart-type de la machine. Chaque poste possède son propre extracteur et sa propre référence. Sans ce cloisonnement, le rythme d'un serveur de fichiers et celui d'un poste bureautique se moyennaient, masquant une attaque sur l'un ou produisant de faux positifs sur l'autre.

Un incident algorithmique a marqué cette phase. Si la machine virtuelle restait inerte pendant l'apprentissage, l'écart-type de la création de fichiers valait zéro. Diviser ensuite un pic de soixante fichiers par une constante minuscule produisait des z-scores de plusieurs millions et faisait échouer le calcul. L'équipe a imposé un plancher d'écart-type égal à 1,0. Ce correctif, d'apparence triviale, a rendu le système utilisable en laboratoire comme sur un poste réellement calme.

Une fois la baseline verrouillée, chaque nouvelle fenêtre est évaluée par z-score. Un écart supérieur à trois écarts-types σ , sous hypothèse gaussienne, une très faible probabilité d'appartenir au régime normal. Ce n'est pas encore une décision d'alerte : c'est une matière première pour le moteur de règles et pour le modèle.

e. Moteur heuristique de detection

Le principe de défense en profondeur interdit de s'en remettre à un seul détecteur. Le moteur heuristique, déterministe, attribue des points de risque à partir de règles lisibles par un analyste. Une création massive de fichiers associée à un z-score élevé ajoute trente points. Une suppression massive en ajoute trente autres. Une entropie de noms supérieure à 5,0 en ajoute quarante. Un comportement de processus enfants suspects en ajoute vingt. Le score est ensuite normalisé sur une échelle de 0 à 1, puis comparé à un seuil.

Le seuil initial, fixé à 0,80, s'est révélé trop élevé. Sysmon ne remonte pas toujours les suppressions (Event ID 23) lorsque windows passe par la corbeille. Un simulateur qui chiffre sans supprimer n'obtenait alors que les points de création et d'entropie, soit 0,70, juste sous le radar. L'équipe a abaissé le seuil à 0,70. Cette calibration, documentée, illustre un enseignement du métier : un détecteur se règle sur des traces réelles, pas sur l'intuition du jour où la règle a été écrite.

Le moteur heuristique sert aussi de filet de sécurité. Si le modèle d'apprentissage automatique classe à tort une variante jamais vue comme normale, l'explosion statistique des z-scores peut encore faire basculer le score au-delà du seuil et déclencher la riposte. Inversement, les règles seules généreraient des faux positifs lors d'une mise à jour windows légitime. D'où l'architecture hybride.

f. Apprentissage automatique (Random Forest et LSTM)

La phase de modélisation visait à dépasser des seuils rigides en apprenant des combinaisons de caractéristiques. Deux familles ont été explorées : une forêt aléatoire (random forest) et un réseau de neurones récurrent de type LSTM.

Le principal obstacle n'a pas été l'algorithme, mais les données. Les jeux publics tels que stratosphere IPS ou UWF-ZeekData22 décrivent surtout le réseau. Ils ne contiennent pas les event ID sysmon de création et de suppression de fichiers, pourtant centraux pour un rançongiciel. L'équipe a donc construit un générateur hybride : agrégation temporelle de

capture réseau en fenêtres de dix secondes, puis injection synthétique d'une empreinte disque (volume de fichiers, entropie, processus enfants).

Un premier entraînement sur un profil unique d'attaque « complète » a produit un modèle aveugle. Un simulateur furtif, qui chiffrait hors ligne sans supprimer ni appeler un serveur de commande, était classé normal. Trois profils ont alors été introduits : une attaque APT complète, un profil hors ligne sans connexions, et un profil sans suppressions. Le jeu final compte 14 874 lignes, dont 555 exemples de rançongiciels mutants.

La forêt aléatoire, cent arbres, a obtenu un F1-score de 1,00 sur le jeu de test, au même titre que le LSTM. L'importance des variables a montré que la profondeur de l'arbre des processus pesait environ 20 %, le volume de fichiers créés 18 %, puis à égalité la diversité des extensions, les renommages et l'entropie, autour de 15 % chacun. Les caractéristiques réseau ressortaient à une importance nulle : le dataset synthétique n'y était pas discriminant. Le moteur heuristique continue donc de les pondérer, ce que le modèle seul ne fait pas.

Le LSTM a été écarté en production. Dans le pipeline, le modèle ne reçoit jamais la séquence brute des événements, mais un vecteur déjà agrégé sur dix secondes : la mémoire temporelle fine que le LSTM sait exploiter a été effacée en amont. À performances égales, il imposait pytorch, soit plus de 2,5 Go de dépendances. L'équipe a préféré une inférence légère, explicable, dont l'importance des variables est lisible par un analyste. Les dépendances de recherche ont été isolées dans un fichier distinct, ce qui a ramené la construction de l'image docker de plus de quinze minutes à environ une minute.

g. API, réponse active et agent de riposte

Détecter sans pouvoir agir laisse le chiffrement se poursuivre. L'équipe a donc conçu une architecture à trois étages, inspirée des EDR d'entreprise : un capteur sur le poste, un cerveau central, une console pour l'humain.

L'API FastAPI reçoit les lots Winlogbeat, orchestre parseur, extracteur, baseline, règles et modèle, et gère une file de commandes par machine. L'extracteur ne se contente plus d'un score global : il suit chaque identifiant de processus. À la fin d'une fenêtre, le processus le plus suspect est identifié, avec son parent, pour reconstituer une chaîne de causalité. Un score inférieur à 50 ne produit qu'un journal. Entre 50 et 80, l'alerte est critique. Au-delà de 80, un ordre d'arrêt est placé dans la file, avec un niveau de confiance élevé.

L'agent powershell, exécuté en administrateur sur la machine virtuelle, interroge l'API toutes les deux secondes. Ce modèle d'appel sortant évite d'ouvrir un port d'administration sur le poste. L'agent n'exécute que les ordres destinés à sa propre machine, lance stop-process sur l'identifiant visé, peut isoler le poste via le pare-feu windows en laissant le canal vers l'EDR et acquitte l'exécution. Une commande non acquittée expire au bout de quinze minutes, afin qu'un poste éteint ne laisse pas un ordre en attente indéfinie.

Le simulateur de démonstration enchaîne trois techniques MITRE ATT&CK : une simulation de protocole applicatif vers un serveur de commande (T1071), une interrogation des clichés de restauration via vssadmin (T1490) et une génération massive de fichiers à haute entropie (T1486). Dans les tests documentés, l'attaque lancée sous un identifiant de processus donné était interceptée par sysmon, encapsulée par winlogbeat, figée dans la fenêtre de dix secondes, confirmée par les deux détecteurs, puis stoppée par l'agent en un peu plus de dix secondes. Un rapport json d'investigation est archivé pour l'analyste.

h. Console SOC multi-analystes et déploiement

La dernière phase technique a transformé un démonstrateur mono-poste en plateforme de supervision. L'exigence de départ, plusieurs analystes doivent voir les mêmes données au même instant a révélé que l'itération précédente n'était pas un produit inachevé, mais une architecture inadaptée.

SQLite verrouillait toute la base à chaque écriture, rendant incompatibles l'ingestion continue et la consultation. L'état de détection vivait en mémoire : un redémarrage effaçait la baseline. Le rôle de l'analyste était décidé par le navigateur, ce qui permettait d'obtenir les droits d'un responsable SOC en modifiant une variable côté client. Le journal d'audit acceptait l'identité fournie par l'appelant. Le score de gravité n'était pas borné et affichait 146 ou 241 sur une échelle présentée comme sur 100. La file de commandes n'avait pas de destinataire : la première machine à interroger l'API récupérait l'ordre destiné à une autre.

La migration vers PostgreSQL 16, avec neuf tables versionnées par Alembic, a levé le verrouillage et rendu déterministe l'agrégation temporelle des graphiques. L'authentification repose désormais sur un hachage argon2id, une session serveur dans un cookie HttpOnly, un verrouillage après cinq échecs, une rotation imposée du mot de passe d'amorçage, et trois niveaux d'habilitation appliqués sur chaque route : n1 consulte et qualifie, n2 peut arrêter un processus et isoler un poste, n3 gère les comptes, les exclusions et la configuration. Un appel direct à la route d'arrêt avec un compte n1 reçoit un refus 403, journalisé.

La synchronisation temps réel utilise un websocket authentifié qui ne transporte aucune donnée métier, seulement un avis d'invalidation par canal. Chaque console relit ensuite l'API. Un message perdu ne peut donc pas laisser deux analystes avec des états divergents, et les droits sont réévalués à chaque lecture. Si le canal tombe, l'interface le signale et bascule sur un rafraîchissement périodique, plutôt que d'afficher silencieusement des données figées.

La console, développée avec react, vite et tailwind css, propose onze onglets : vue d'ensemble, terminaux, alertes, journal des réponses, statistiques du modèle, moteur heuristique, exclusions, journal d'audit, équipe SOC, configuration et documentation embarquée. Aucun indicateur n'est calculé dans le navigateur. Le déploiement docker compose up démarre la base, l'API et le frontal. Un reverse proxy unique évite les écueils de CORS et permet au cookie de session de fonctionner pour un analyste distant. L'API tourne volontairement avec un seul processus de calcul : les extracteurs sont des automates à état en mémoire ; les répartir entre plusieurs workers fausserait les fenêtres.

i. Tests et validation du pipeline

La validation n'a pas reposé uniquement sur des captures d'écran. Trois suites automatisées ont été exécutées contre le déploiement conteneurisé complet, donc à travers le reverse proxy, et non seulement en environnement de développement.

Suite	Portée	Résultat
scripts/e2e_check.py	86 contrôles : authentification, habilitations, ingestion, détection, réponse, temps réel, audit	86 / 86
scripts/ui_check.py	37 contrôles : parcours navigateur à travers le proxy, cookie de session compris	37 / 37
dashboard/tests/smoke.mjs	Rendu des 11 onglets dans Chromium, échec à la moindre erreur JavaScript	11 / 11

Ces suites vérifient notamment que deux analystes obtiennent des chiffres identiques, qu'un compte n1 ne peut pas déclencher d'arrêt de processus, que le cookie de session est invisible du javascript dans un vrai navigateur, et que la dernière fenêtre d'une attaque est analysée même lorsque l'agent cesse d'émettre. Cette dernière propriété corrige un angle mort : si le rançongiciel éteint le poste juste après son passage, aucun événement ultérieur n'arrive

pour « fermer » la fenêtre qui contient pourtant la preuve. Une tâche de fond évalue donc les fenêtres restées inactives.

La démonstration live, répétée, enchaîne l'exécution du simulateur sur la machine virtuelle, l'apparition de l'alerte sur toutes les consoles connectées, le passage de la commande d'arrêt de pending à acked, et l'arrêt visible du processus dans le terminal de l'agent. Le snapshot VMware permet ensuite de revenir à un état propre. Cette chorégraphie, d'apparence simple, condense quelques semaines d'intégration.

j. Bilan des missions

Résultats obtenus et valeur ajoutée

Le pipeline déployé répond aux objectifs fixés. Un poste windows surveillé par sysmon remonte des événements authentifiés. Le serveur en déduit un vecteur de comportement, compare à une baseline locale, croise règles et forêt aléatoire, crée une alerte bornée sur 100, et peut arrêter le processus responsable. Plusieurs analystes voient la même réalité, avec des droits distincts et un journal d'audit que l'appelant ne peut pas falsifier. Le temps entre le début du simulateur et l'arrêt du processus se compte en secondes, là où un traitement manuel d'alerte prendrait plusieurs minutes.

Pour le bachelor, le livrable n'est pas seulement un code source. C'est une architecture documentée, un guide de reproduction destiné à un non-développeur, des tests rejouables pendant la soutenance, et une démonstration confinéesans malware réel. L'usage pédagogique a été rappelé dans le dépôt : aucune donnée réelle de rançongiciel n'est utilisée.

Réflexions critiques

Avec le recul, plusieurs aspects auraient pu être abordés autrement. Le dataset d'entraînement, en grande partie synthétique, explique des métriques parfaites sur le jeu de test. Un F1-score de 1,00 n'est pas une preuve de généralisation au monde réel ; c'est la trace d'une séparation trop nette entre le bruit de fond généré et l'empreinte injectée. Les caractéristiques réseau, d'importance nulle dans le modèle, auraient mérité des captures sysmon event ID 3 plus riches, plutôt qu'une greffe statistique. La première console, trop confiante dans le navigateur, a coûté une phase entière de reprise : mieux aurait valu poser les sessions serveur et PostgreSQL dès le début.

La répartition à trois a parfois ralenti l'intégration. Un contrat d'interface plus tôt sur le format des commandes agent aurait évité que la file d'ordres se comporte comme une queue

globale. À l'inverse, le travail collectif a empêché qu'un seul membre porte à la fois la détection, l'interface et le laboratoire, ce qui aurait produit un démonstrateur plus fragile.

Axes d'amélioration et préconisations

Plusieurs évolutions se dessinent. Enrichir le dataset avec des traces sysmon réelles de familles publiques en laboratoire isolé renforcerait le modèle. Intégrer une source de réputation d'indicateurs (sous réserve juridique et de confidentialité) réduirait les faux positifs réseau. Exposer davantage de télémétrie sysmon y compris des événements aujourd'hui filtrés améliorerait la visibilité sur des techniques furtives. Passer `app_env` à production et activer `cookie_secure` derrière une terminaison TLS est un préalable à toute exposition hors du laboratoire. Enfin, une authentification mutuelle plus forte des agents, au-delà d'un jeton partagé, rapprocherait la solution des EDR commerciaux.

III- CONCLUSION ET PERSPECTIVES

À la date de finalisation de ce rapport, les objectifs fixés en début de projet de substitution de stage sont atteints. La solution détecte des comportements de rançongiciel à partir d'événements sysmon, décide par un croisement heuristique et statistique, répond sur le poste via un agent powershell, et offre à plusieurs analystes une console partagée, authentifiée et auditable. Les tests automatisés du déploiement docker (86 contrôles de bout en bout, 37 contrôles d'interface, 11 écrans de la console) et la démonstration sur machine virtuelle en apportent la preuve opératoire.

Le pipeline repose sur une architecture cohérente. Sysmon et winlogbeat collectent. Le parseur normalise. L'extracteur fige des fenêtres de dix secondes. La baseline apprend le rythme de chaque machine. Le moteur de règles et la forêt aléatoire se contrôlent l'un l'autre. L'API archive, notifie et ordonne. L'agent exécute et acquitte. PostgreSQL est la vérité partagée. Le navigateur n'est qu'une vue contrainte par des habilitations serveur.

Au-delà des résultats techniques, cette expérience nous a confronté à la réalité d'un projet de cybersécurité opérationnelle : passer des heures sur un jeton refusé, découvrir qu'une base SQLite ne tient pas la charge concurrente, accepter qu'un modèle au score parfait soit d'abord le reflet de ses données, et documenter pour quelqu'un qui n'a pas écrit le code. On acquies une maîtrise concrète de la collecte windows, du feature engineering, de FastAPI, de PostgreSQL, de react et de docker, ainsi qu'une première expérience de conception d'une console SOC.

Les perspectives à court terme concernent le durcissement hors laboratoire et l'enrichissement du jeu d'entraînement. À moyen terme, des mécanismes de réponse plus fins et une télémétrie plus large rapprocheraient encore le prototype des produits du marché. Le livrable reste néanmoins ce qu'il doit être dans le cadre de l'ECE : un projet pédagogique, reproductible, éthiquement confiné, qui conforte l'orientation des trois membres vers les métiers de la blue team, de l'analyse SOC et de l'ingénierie EDR.

IV- BIBLIOGRAPHIE / WEBOGRAPHIE

Documentation officielle des outils utilisés

- Microsoft Sysinternals. Documentation Sysmon. Disponible sur : <https://learn.microsoft.com/sysinternals/downloads/sysmon>
- Elastic. Documentation Winlogbeat 8.18. Disponible sur : <https://www.elastic.co/guide/en/beats/winlogbeat/current/index.html>
- FastAPI. Documentation officielle. Disponible sur : <https://fastapi.tiangolo.com>
- PostgreSQL Global Development Group. Documentation PostgreSQL 16. Disponible sur : <https://www.postgresql.org/docs/16/>
- Docker Inc. Documentation Docker Compose. Disponible sur : <https://docs.docker.com/compose/>
- scikit-learn. User Guide — Ensemble methods (Random Forest). Disponible sur : <https://scikit-learn.org>
- React / Vite. Documentation officielle. Disponibles sur : <https://react.dev> et <https://vitejs.dev>

Référentiels et frameworks de cybersécurité

- MITRE Corporation. MITRE ATT&CK Framework. Techniques T1071, T1486, T1490. Disponible sur : <https://attack.mitre.org>

- NIST. Guide to Intrusion Detection and Prevention Systems (IDPS). NIST SP 800-94. Disponible sur : <https://csrc.nist.gov>
- ANSSI. Recommandations relatives à la supervision de sécurité. Disponible sur : <https://www.ssi.gouv.fr>
- ISO/IEC 27001:2022. Systèmes de management de la sécurité de l'information.

Jeux de données et sources académiques

- Stratosphere Laboratory, CTU Prague. Stratosphere IPS datasets. Disponible sur : <https://www.stratosphereips.org>
- University of West Florida. UWF-ZeekData22.
- Shannon, C. E. A Mathematical Theory of Communication. Bell System Technical Journal, 1948.
- Breiman, L. Random Forests. Machine Learning, 2001.

Établissement et cadre de formation

- ECE Paris. Présentation de l'école et du Bachelor Cybersécurité & réseaux. Disponible sur : <https://www.ece.fr>
- ECE / OMNES Education. Démarche RSE Time To Act. Disponible sur : <https://www.ece.fr/rse/>
- ECE Paris. Campus de Paris, 10 rue Sextius Michel, 75015 Paris. Disponible sur : <https://www.ece.fr/nos-campus/campus-de-paris/>

V- ANNEXES

Les annexes ci-après complètent les éléments techniques du rapport. Elles décrivent l'architecture, les onglets de la console et le déroulé de la démonstration. Les captures d'écran de la soutenance pourront y être insérées par l'équipe.

Annexe 1 : Schéma d'architecture de la plateforme

Le système se répartit entre des postes Windows surveillés et un serveur EDR conteneurisé. Sur chaque poste : Sysmon capture, Winlogbeat expédie, l'agent PowerShell interroge la file de commandes et exécute arrêt ou isolation. Sur le serveur : nginx expose une origine unique pour la console, l'API et le WebSocket ; FastAPI enchaîne parseur, extracteur cloisonné par machine, baseline, règles, forêt aléatoire et moteur de réponse ; PostgreSQL conserve comptes, sessions, alertes, métriques, commandes, exclusions, journal d'audit et paramètres. Les analystes N1, N2 et N3 s'authentifient par cookie de session.

Flux principal : Sysmon → Winlogbeat → POST /ingest (jeton d'agent) → détection hybride → alerte en base → avis WebSocket → consoles. Flux de riposte : API → file commands → agent → Stop-Process ou netsh advfirewall → accusé de réception.

Annexe 2 : Vecteur de caractéristiques (fenêtre de 10 secondes)

Feature	Origine principale	Rôle pour la détection
nb_files_created	Sysmon 11	Pic de chiffrement
nb_files_deleted	Sysmon 23	Effacement des originaux
nb_files_renamed	Sysmon	Renommage massif
nb_unique_extensions	Sysmon 11	Diversité des cibles
entropy_filenames	Noms de fichiers	Aléatoire cryptographique
nb_processes_created	Sysmon 1	Activité processus
nb_child_processes	Sysmon 1	Lancement d'outils (vssadmin, cmd)
process_depth	Sysmon 1	Profondeur d'arbre, souvent discriminante
nb_connections	Sysmon 3	Volume réseau
nb_unique_ips	Sysmon 3	Dispersion des destinations
nb_external_connections	Sysmon 3	Sortie hors RFC 1918

nb_dns_queries	Sysmon 22 (si présent)	Résolution de domaines
----------------	------------------------	------------------------

Annexe 3 : Onglets de la console SOC

Onglet	Usage
Dashboard	Compteurs, graphique partagé, file d'alertes
Terminaux	Postes qui envoient des logs, phase apprentissage ou détection
Alertes de sécurité	Journal des détections et fiche d'enquête
Journal des réponses	Ordres d'arrêt et d'isolation, pending puis acked
Statistiques ML	Modèle chargé et importances
Moteur heuristique	Règles et seuils réellement en vigueur
Règles d'exclusion	Chemins de confiance — réservé N3
Journal d'audit	Qui a fait quoi, non falsifiable par le client
Équipe SOC	Création de comptes — réservé N3
Configuration	Seuils d'alerte et de kill — réservé N3
Documentation	Aide intégrée, sans quitter la console

Annexe 4 : Scénario de démonstration

Prérequis : Docker Compose démarré sur l'hôte, machine virtuelle Windows avec Sysmon, Winlogbeat et agent PowerShell, snapshot propre pris, analyste connecté en N2 ou N3 sur <http://localhost:8080>.

- Sur la VM, exécuter `simulate_ransomware_v2.ps1` depuis le dossier agent.
- Observer l'alerte sur toutes les consoles ouvertes, sans rechargement manuel.
- Vérifier dans le journal des réponses le passage de pending à acked.
- Constater l'arrêt du processus dans le terminal de l'agent.
- Ouvrir la fiche d'alerte : vecteur de la fenêtre, processus parent, score borné.
- Restaurer le snapshot VMware pour un état propre.

Annexe 5 : Répartition interne des rôles

Axe	Responsabilité principale	Modules
M1 — Collecte et pipeline	Un membre du groupe	Lab, Sysmon, Winlogbeat, parseur, features, baseline
M2 — Détection	Un membre du groupe	Règles, dataset, Random Forest, LSTM, calibration

M3 — Plateforme	Un membre du groupe	API, agent de riposte, console, PostgreSQL, Docker
Transversal	Les trois membres	Tests E2E, documentation, démonstration, rapport

Les noms nominatifs associés à M1, M2 et M3 sont à reporter en page de garde et, le cas échéant, dans cette annexe avant le dépôt du rapport.

ÉVALUATION DU PROJET DE SUBSTITUTION DE STAGE

Évaluation du comportement

Critère	0	1 Insuffisant	2 Acceptable	3 Bien	4 Très bien
Insertion dans le cadre académique et respect du dispositif de substitution					
Sens du relationnel (tuteur, binômes, soutenance)					
Esprit d'équipe (répartition M1 / M2 / M3, revues croisées)					
Autonomie (laboratoire, débogage, choix techniques)					
Initiative (hybridation ML, reprise PostgreSQL, tests auto)					
Respect des normes (éthique lab, pas de malware réel, secrets hors Git)					

Sous-total / 24

Évaluation des compétences professionnelles

Critère	0	1	2	3	4
Planification et organisation (phases 1 à 7,					

jalons)					
Contrôle et rigueur (seuils, permissions, tokens)					
Reporting (docs techniques, README, rapport)					
Compréhension du cadre ECE / métier SOC-EDR					
Maîtrise des méthodes (Git, Docker, tests, lab VMware)					
Qualité de communication écrite et orale					

Sous-total / 24

Évaluation des compétences techniques

Critère	0	1	2	3	4
Acquisition des connaissances (Sysmon, ML, API, SOC)					
Organisation personnelle et collective du travail					
Maîtrise de l'environnement (Windows, Docker, réseau lab)					
Qualité technique des travaux (détection hybride, console,					

sécurité)					
Respect des délais du planning de substitution					

Sous-total / 20

Total / 68

Commentaires de l'encadrant :

Signature du tuteur académique :